ИТМО

# Docker

# Docker

ИТМО

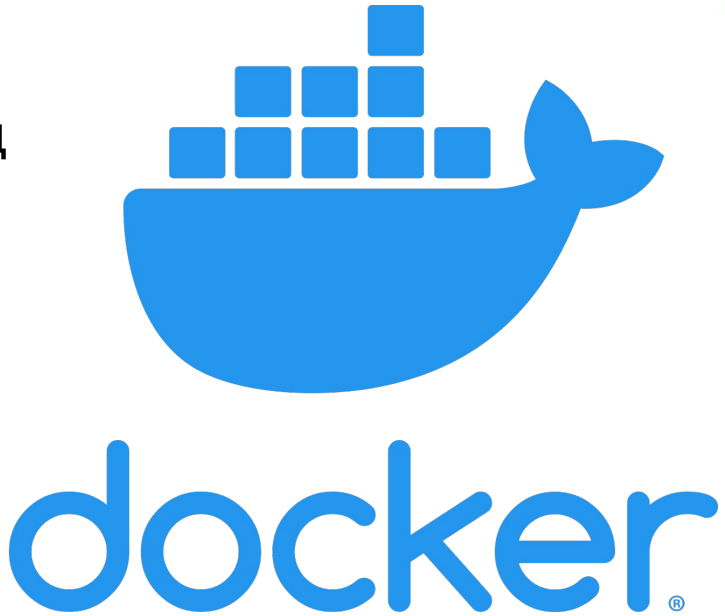
Docker — программное обеспечение для автоматизации развёртывания и управления приложениями в средах с поддержкой контейнеризации, контейнеризатор приложений.

Docker упаковывает приложения со всем его окружением в ОБРАЗ и запускает его на любой машине как изолированный процесс - КОНТЕЙНЕР

# Решаемые проблемы

ИТМО

- Переносимость проекта
- Простота установки
- Конфликт зависимостей и тд



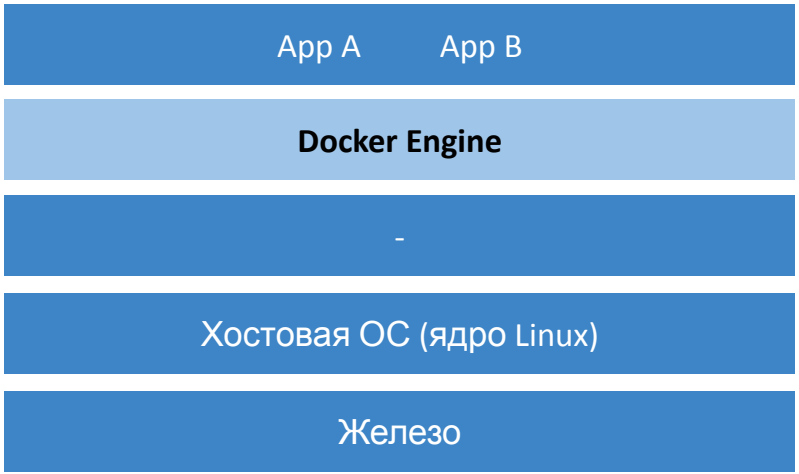
# Виртуализация vs Контейнеризация

ИТМО

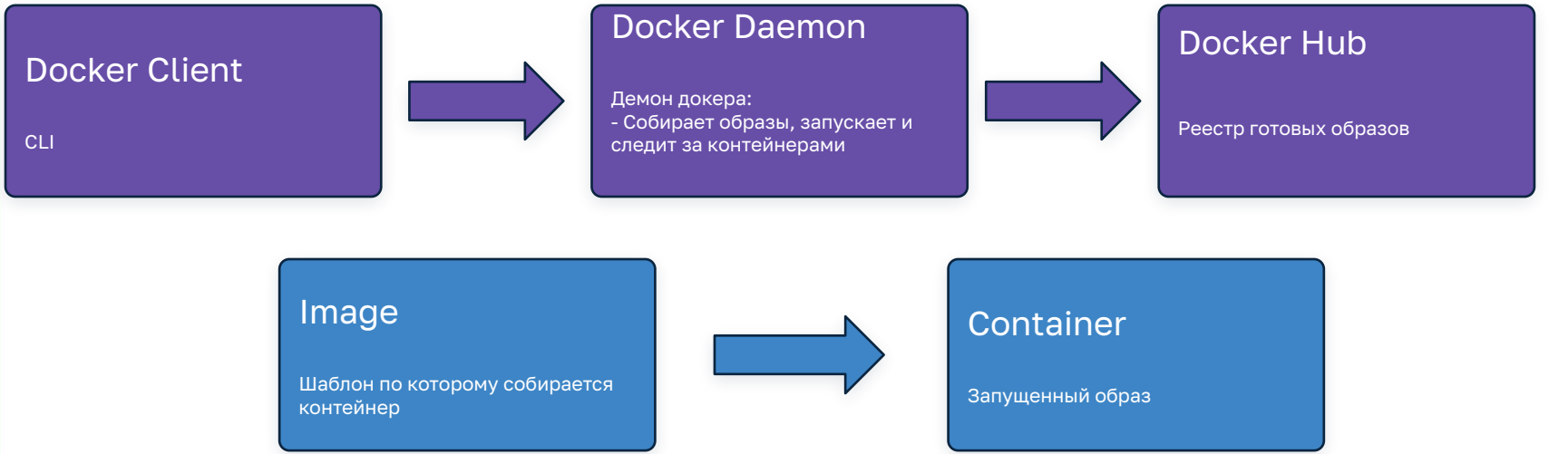
Виртуализация




Контейнеризация



# Терминология



# Пример Dockerfile

ИТМО

```dockerfile
FROM python:3.12-slim

# рабочая директория внутри образа
WORKDIR /usr/src/app

# сначала зависимости — ради кэша
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

# потом остальной код
COPY . .

EXPOSE 5000
CMD ["python", "./app.py"]
```

| | |
|---|---|
| FROM | базовый образ, от которого наследуемся |
| WORKDIR | рабочая папка для всех следующих команд |
| COPY | кладём файлы с хоста внутрь образа |
| RUN | выполняется при сборке образа |
| EXPOSE | документирует порт (сам не публикует) |
| CMD | выполняется при старте контейнера |

# Volumes

ИТМО

Всё, что контейнер пишет в свой writable-слой, удаляется вместе с ним. База, загрузки пользователей, кэш модели НЕОБХОДИМО перекладывать в volume чтобы это просто не терялось при перезапуске контейнера.

Named-volume:

```
run     -v pgdata:/var/lib/postgresql/data
```

Bind mount:

```
run     -v ./web:/opt/app
```

# Кеширование слоей

ИТМО

```
FROM python:3.12-slim

# рабочая директория внутри образа
WORKDIR /usr/src/app

# сначала зависимости — ради кэша
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

# потом остальной код
COPY . .

EXPOSE 5000
CMD ["python", "./app.py"]
```

Каждая новая команда создает новый слой, который в дальнейшем Докер переиспользует пока не изменились его выходные данные.

# CLI Шпаргалка

ИТМО

| | |
|---|---|
| docker pull python:3.12-slim | скачать образ из реестра |
| docker build -t myapp <путь до докерфайла> | сбилдить образ по докер файлу |
| docker run -p 5000:5000 myapp | запустить приложение по тегу образа |
| docker ps / docker ps -a | посмотреть запущенные контейнеры/все контейнеры |
| docker logs -f <id> | посмотреть логи контейнера в реальном режиме |
| docker exec -it <id> bash | зайти внутрь работающего контейнера |

# Docker-compose

ИТМО

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data

  web:
    build:
      context: ./web
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgres://postgres:secret@db:5432/app
    depends_on:
      - db

volumes:
  pgdata:
```

| | |
|---|---|
| services | каждый сервис (отдельный контейнер) |
| ports | паблишинг портов (хост : контейнер) |
| depends_on | выстроенная зависимость одного контейнера от других |